음성 AI 전문가 과정 · Day 21 / 40

Day 21

Voice Agent 아키텍처
Cascade vs S2S · Latency Budget

```
cascade_v1(user_audio)
  endpoint 0.51  stt 0.3  llm 1.4  tts 0.1
  v2v = 2.3 s

cascade_v2(user_audio)   # streaming
  ttft 0.15  first_sentence 0.5
  v2v = 1.4 s
min_silence 300ms → ✂ 잘림 · 800ms → 0.80 s 대기
```

출력 모양 예시 · 숫자는 각자 실행 결과

지형도 → Cascade v1 (VAD · STT · LLM · TTS) → 시간 예산 · p95 → Streaming v2 → VAD Endpointing → 아키텍처 선택
5 주차 M5 시작 · 4 시간 · T4 GPU · API 키 불필요 · 실습 노트북 Part 0 ~ 7

0.0 · 오늘 답할 질문

음성 비서는 왜 " 빨리 말하기 시작 " 해야 할까 ?

1 음성 비서는 어떤 부품으로 이루어지고 , Cascade 와 S2S 는 무엇이 다른가 ? Part 1

2 한 턴에서 어느 단계가 시간을 가장 많이 쓰나 ? Part 2

3 목표 지연을 단계별로 어떻게 나누고 , 무엇으로 판정하나 ? Part 3

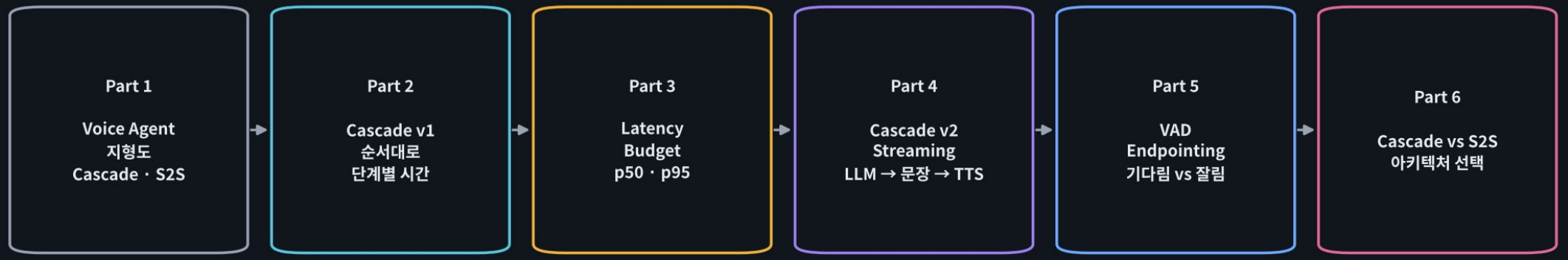
4 LLM 답을 다 기다리지 않고 말하기 시작하려면 ? Part 4

5 " 말이 끝났다 " 는 언제 판단하고 , 무엇과 맞바꾸나 ? Part 5

6 내 서비스에는 어떤 구조가 맞나 ? Part 6

잇고 · 재고 · 줄이고 · 고른다

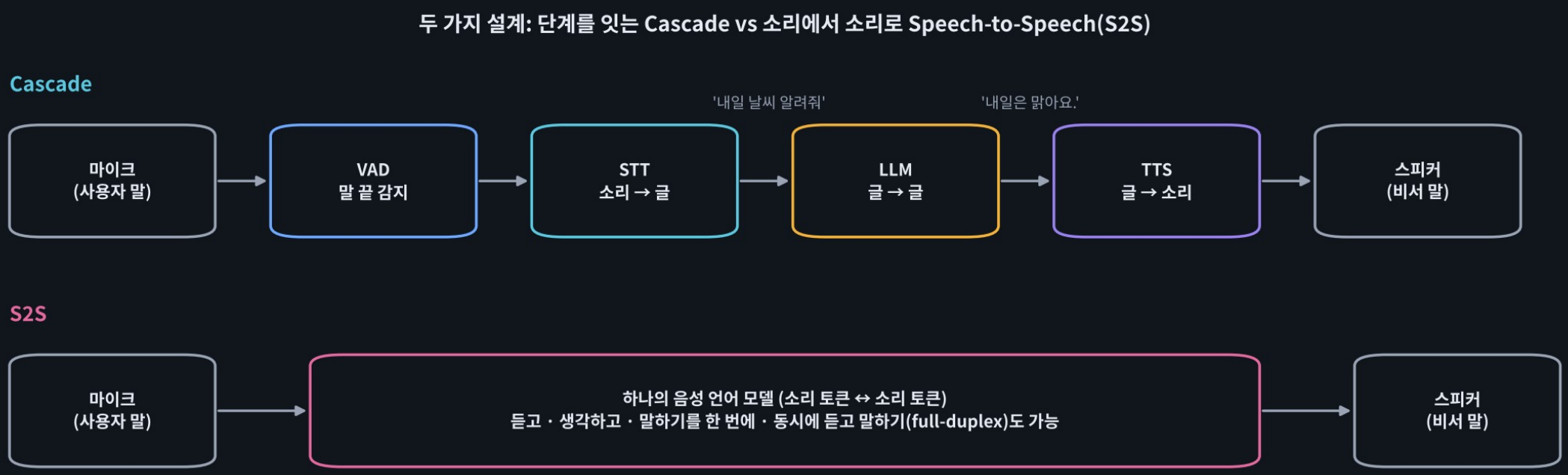
D21 · 5주차 시작: 귀(ASR) + 두뇌(LLM) + 입(TTS) = 음성 비서



오늘의 모델: Silero VAD(D4) · Whisper(D7) · Qwen3 소형 LLM(로컬, API 키 불필요) · MMS-TTS 한국어 VITS(D16)

교시	내용	빈칸	애니메이션
1 교시	Part 0 ~ 2 지형도 · 부품 · Cascade v1	1 ~ 3	D21-1 · 2 · 5 · 6
2 교시	Part 3 Latency Budget · p50 · p95 · 답 길이	4 · 5	D21-3 · 7 · 8
3 교시	Part 4 ~ 5 Streaming v2 · VAD Endpointing	6 ~ 8	D21-4 · 5
4 교시	Part 6 ~ 7 Cascade vs S2S 선택 · 정리	9	D21-9

Cascade (부품 잇기) vs Speech-to-Speech



Cascade: 중간 글을 보고 고칠 수 있고 부품을 바꿀 수 있음 · 지연이 더해짐 | S2S: 빠르고 억양 · 감정이 살아 있음 · 속을 들여다보기 어렵고 공급사에 묶임

▶ 애니메이션 D21-2 Cascade vs S2S 흐름

🍎 통역사 셋 vs 이중 언어 화자

쪽지 (글) 가 남으면 확인은 쉽지만 넘기는 시간이 든다 .

사람은 0.2 ~ 0.3 초 , 비서는 ?

지연 (말 끝 → 첫 소리)	느낌 (업계 보고 예)
0.2 ~ 0.3 초	사람끼리 대화의 틈
~ 0.8 초	자연스러운 비서
0.8 ~ 1.2 초	기다림이 느껴짐
1.2 초 초과	" 여보세요 ?" · 겹쳐 말하기

▶ 애니메이션 D21-6 턴 테이킹

📐 핵심 식

voice-to-voice
= 말 끝 감지 + STT
+ LLM(첫 문장) + TTS(첫 소리)
+ 네트워크

네 부품 · 첫 호출은 느리다

부품	모델	배운 날
VAD	Silero VAD (VADIterator, 32 ms)	D4
STT	Whisper small / tiny	D7 · D8
LLM	Qwen3-1.7B / 0.6B · 생각 모드 끔	새로
TTS	MMS-TTS 한국어 (VITS)	D16

🍎 자동차 예열

첫 손님만 느린 서버 → 켤 때 한 번 돌려 두기 (warm-up)

```
부품  첫_호출  두번째  배
STT    2.4      0.3    8x
LLM    1.9      0.4    5x
```

예시 숫자 · 각자 실행 결과로

감지 시각 ≈ 말 끝 + 침묵 기준

```
@contextmanager
def stage(T, name):
    t_begin = time.perf_counter(); yield; sync()
    T[name] = time.perf_counter() - ____     # 1

it = VADIterator(vad, sampling_rate=SR,
        min_silence_duration_ms=____)         # 2

[{"role": "____", "content": VOICE_SYSTEM},  # 3
 {"role": "user", "content": heard}]
apply_chat_template(..., enable_thinking=False)
```

🧩 빈칸 실습 1 · 2 · 3

1 → t_begin · 2 → min_silence_ms

3 → "system"

음성용 시스템 지시

- 한두 문장 , 60 자 안쪽
- 구어체
- 목록 · 표 · 마크다운 · 이모지 금지
- 모르면 짧게 모른다고

순서대로 처리하면 1.5 ~ 3 초

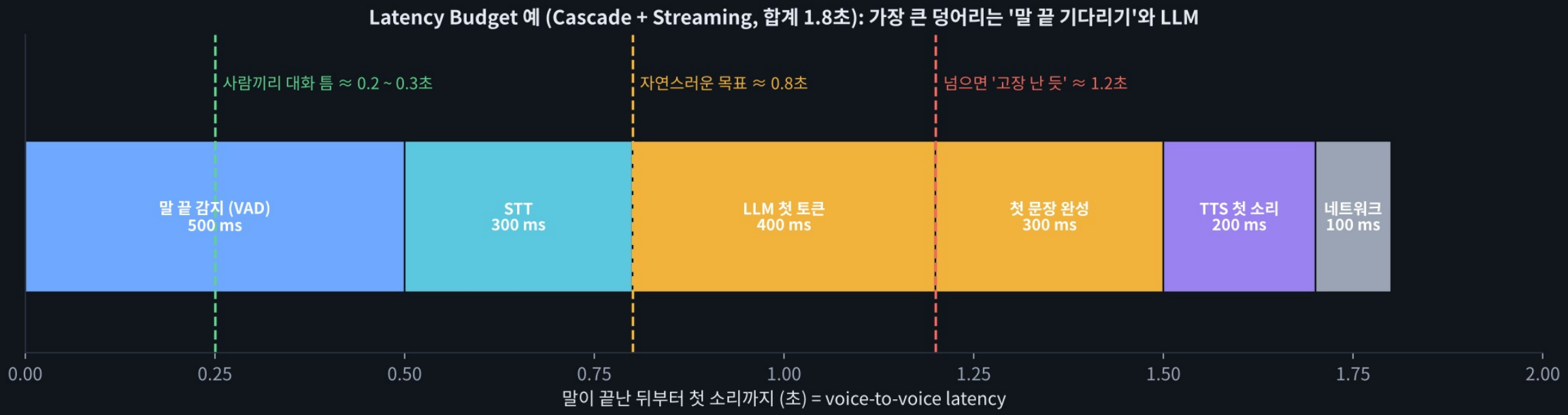
1. VAD: 첫 " 말 끝 " 에서 턴 종료
2. 그때까지 소리를 STT
3. LLM: 시스템 지시 + 이력 + 들은 말
4. TTS → 재생
5. v2v = endpoint + stt + llm + tts

단계	시간 (T4 예)	비중
endpoint	0.51	말 끝 기다림
stt	0.3	
llm	1.4	가장 큼
tts	0.1	
v2v	2.3	> 1.2 초

가장 큰 칸 = LLM(답 전체 기다림) · 그다음 = 말 끝 기다림 → Part 3 · 4 · 5 에서 줄이기

📝 **1 교시 워크시트 → worksheet.html 1 교시 탭 10 문항**

목표를 단계별 한도로 나눈다



1. 목표 : p50 ≤ 1.0 초 , p95 ≤ 1.5 초
2. 단계별 한도 나누기
3. 여러 번 재서 p50 · p95
4. 넘친 단계부터 줄이기

▶ 애니메이션 D21-3 Latency Budget 설계기

평균은 가끔 느린 응답을 숨긴다

```
BUDGET = {"endpoint": .5, "stt": .3, "llm": .5, "tts": .2}
TOTAL = sum(BUDGET.____())              # 4 values
p50 = np.percentile(V2V, 50)
p95 = np.percentile(V2V, ____)          # 5 95
```

🧩 빈칸 실습 4·5

4 → values · 5 → 95

▶ 애니메이션 D21-7 p50 • p95 읽기

단계	예산	측정 p50	판정
endpoint	0.50	0.51	≈
stt	0.30	0.30	✅
llm	0.50	1.40	❌ 넘침
tts	0.20	0.08	✅

판정 예시 (v1, T4)

LLM 시간 ≈ 첫 토큰 + 토큰 수 ÷ 초당 토큰

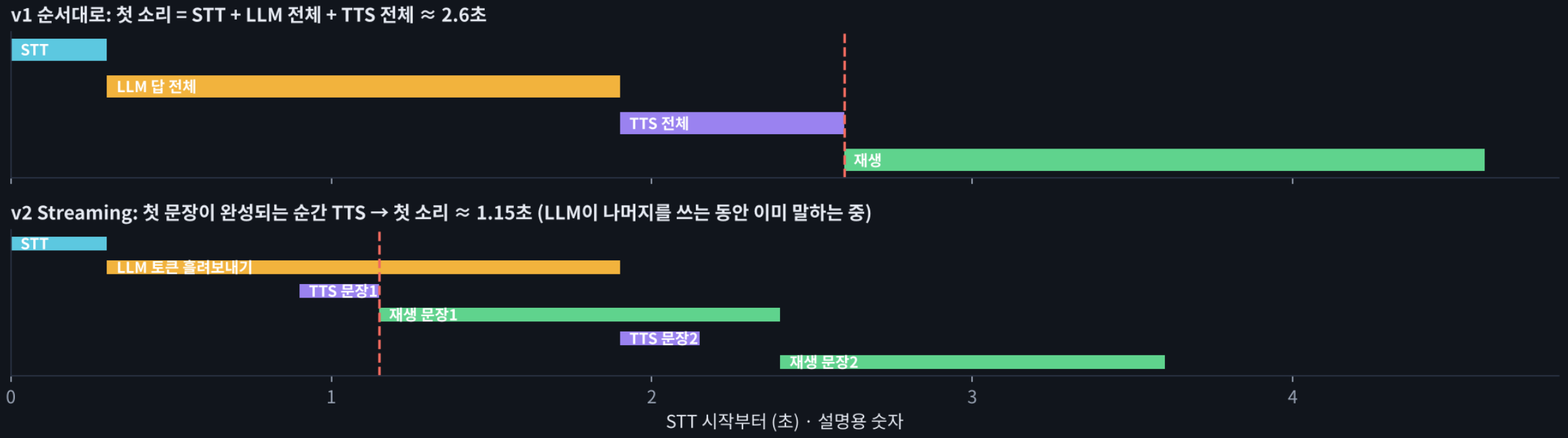
지시	토큰 (예)	LLM 시간 (T4 예)
음성용 (짧게)	20 ~ 50	1 ~ 2 초
글 채팅용 (자세히)	150 ~ 256	5 ~ 9 초
생각 모드 켬	+ 수백	+ 수 초

▶ 애니메이션 D21-8 답 길이 = 지연

📝 2 교시 워크시트 → worksheet.html 2 교시 탭 10 문항

❓ 0.5 초 예산이면 ?
(0.5 − 0.15) × 30 ≈ 10 토큰
→ 한 문장도 어려움
→ 기준을 " 첫 문장 " 으로 (Streaming)

첫 문장이 완성되면 바로 TTS



🍎 **코스 요리**

전채가 되면 바로 낸다 . 손님이 먹는 동안 주방은 본 요리를 만든다 .

▶ 애니메이션 D21-4 v1 vs v2 타임라인

Thread.start() · TTFT · 문장 자르기

```
streamer = TextIteratorStreamer(tok, skip_prompt=True)
th = Thread(target=llm.generate, kwargs=dict(..., streamer=streamer))
th.____()                                # 6 start
for piece in streamer: yield piece

ttft = time.perf_counter() - ____        # 7 t_llm

END = re.compile(r"[.!?。…~]+\s")
while (m := END.search(buf, ____)):   # 8 min_chars
    yield buf[:m.end()].strip(); buf = buf[m.end():]
```

🧩 **빈칸 실습 6·7·8**

6 → start · 7 → t_llm

8 → min_chars

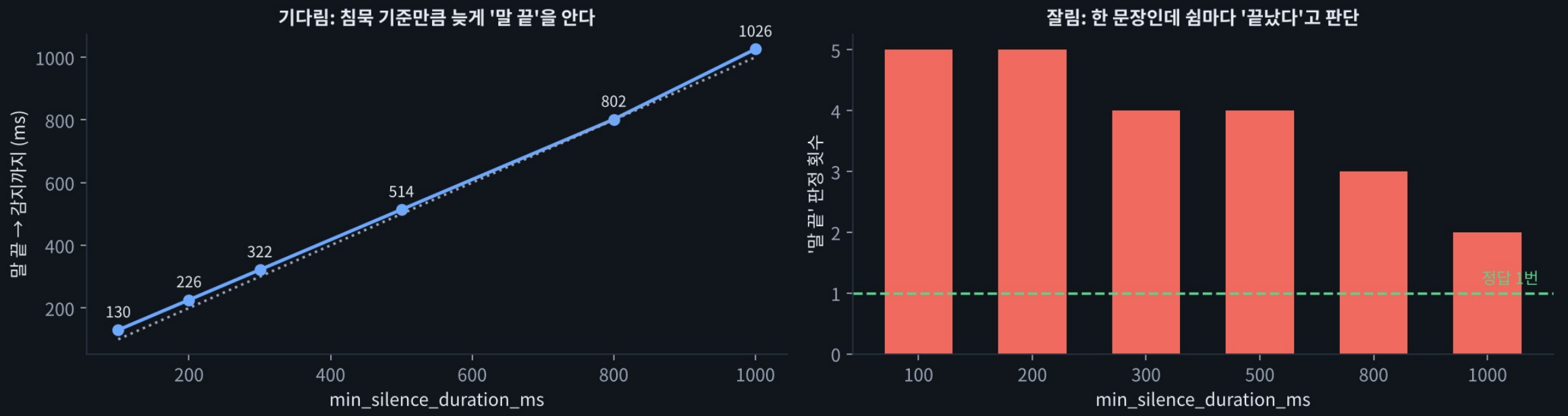
```
[" 안녕하세요 . ", " 네 .  오늘은  ",
 " 맑아요 !  내일도 ", "  좋아요 ."]
→  안녕하세요 .  네 . │ 오늘은 맑아요 !
   │  내일도 좋아요 .
```

여러 문장일수록 이득이 크다

구분	v1 순서대로	v2 Streaming
LLM 칸	답 전체	첫 문장까지 (TTFT + 첫 문장 토큰)
첫 소리 (T4 예)	1.5 ~ 3 초	1 ~ 1.5 초
한 문장 답	—	거의 같음
남는 병목	LLM · 말 끝	말 끝 · STT

❓ **말하는 도중 사용자가 끼어들면? (Barge-in, D28)**
계속 듣는 VAD · 재생 중지 · 생성 중단 · 말한 데까지만 이력에 남기기 · 에코 제거 (AEC)

짧으면 잘리고 , 길면 늦다



강사 실측: Silero VAD 6.2 · JFK 연설 11초(한 문장, 쉼 4번) · 512샘플(32 ms) 단위 · 기다림 ≈ 침묵 기준 + 약 30 ms

🍎 눈치 보는 대화 상대

숨만 쉬어도 말을 받으면 끊고 , 3 초씩 기다리면 답답하다 .

▶ 애니메이션 D21-5 VAD 말 끝 감지

침묵 기준은 대화 상태마다 다르게

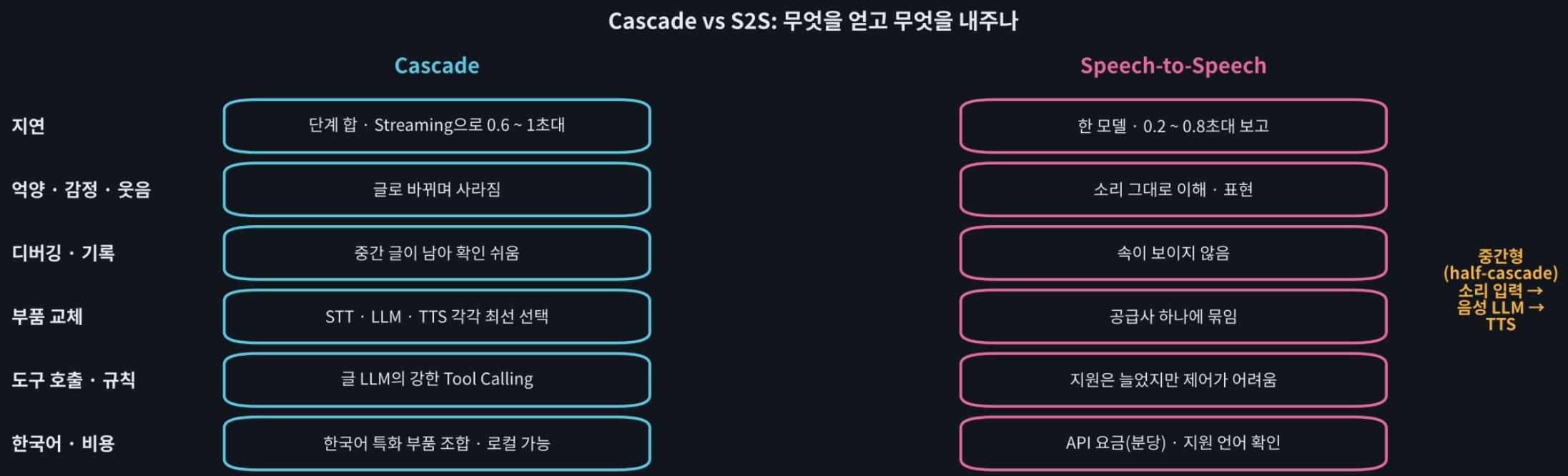
기준 (ms)	첫 판정	결과
200	쉼에서	✂ 잘림
300	쉼에서	✂ 잘림
500	쉼에서	✂ 잘림
800	실제 끝 + 0.80	대기
1000	실제 끝 + 1.03	대기

쉼 0.6초 문장 · 테스트 실행 예

대책	배우는 날
상태별 침묵 기준	D28
Semantic Turn Detection	D28 · D29
미리 준비 (Speculative)	D28
Barge-in	D28

📝 3 교시 워크시트 → worksheet.html 3 교시 탭 10 문항

무엇을 얻고 무엇을 내주나



업계 지연 숫자는 보고 · 측정마다 다름 (2026년 기준 예) · 이번 과정: 5주차 Cascade로 원리 → 6주차 D30에서 S2S API 비교

▶ 애니메이션 D21-9 아키텍처 선택기

max(scores, key=scores.get)

```
scores = {name: sum(w[k] * pts[k] for k in w)
          for name, pts in ARCH.items()}
best = max(scores, key=scores.____)     # 9 get
```

🧩 **빈칸 실습 9**

9 → get

상황	Cascade (로컬)	Cascade (API)	S2S (API)	선택
병원 예약 콜센터	70	65	49	Cascade (로컬)
외국어 회화 튜터	31	33	41	S2S (API)
어르신 말벗	53	51	49	Cascade (로컬) · 경계
공장 현장 음성 보고	67	60	39	Cascade (로컬)

정리 · 오늘 배운 것

" 언제 " 말하느냐도 품질이다

개념	한 줄	코드
시간 재기	시작 시각을 빼고 · GPU 는 sync	`perf_counter() - t_begin`
말 끝 감지	침묵이 기준만큼 이어지면 end	`min_silence_duration_ms`
음성용 지시	짧게 · 구어체 · 생각 모드 끔	`{"role": "system"}`
Latency Budget	단계별 한도 · p50 · p95	`sum(BUDGET.values()), percentile 95`
Streaming	다른 스레드 생성 · 문장마다 TTS	`th.start(), END.search(buf, min_chars)`
TTFT	LLM 시작부터 첫 토큰	`perf_counter() - t_llm`
Endpointing	짧으면 잘림 · 길면 늦음	침묵 기준 실험표
아키텍처 선택	가중치 × 점수 · 민감도	`max(scores, key=scores.get)`

D22: 큰 LLM 을 API 로 연결한다

- ☐ Voice Agent 부품과 Cascade · S2S 차이를 설명할 수 있다
- ☐ Cascade v1 을 만들고 단계별 시간을 쟀다
- ☐ Latency Budget 을 세우고 p50 · p95 로 판정했다
- ☐ Streaming v2 로 첫 소리를 앞당겼다
- ☐ 침묵 기준의 기다림 · 잘림을 실험했다
- ☐ 서비스 기준으로 구조를 골랐다

다음 : D22 LLM API 연동

Claude · OpenAI · Gemini API 를 연결해 답의 품질과 TTFT 를 비교하고 , 음성 대화용 프롬프트 (짧은 응답 · 구어체 · 되묻기) 와 대화 이력 관리를 다룹니다 . API 키를 Colab 보안 비밀에 넣어 오세요 .

`worksheet.html`	40 문항
개념 워크시트 · 노션 제출	
`animation.html`	9 개 탭
인터랙티브 애니메이션	
`D21_VoiceAgent 아키텍처 _ 실습 .ipynb`	빈칸 9
실습 노트북 · T4	

온라인 자료 imguru-mooc.github.io/AI_Voice/day21